

OS practicals

✓ 1(i) Palindrome (Shell)

```
echo "Enter number:"read nrev=0temp=$nwhile [ $temp -gt 0 ]do r=$((temp % 10)) rev=$((rev * 10 + r)) temp=$((temp / 10))doneif [ $n -eq $rev ]then echo "Palindrome"else echo "Not Palindrome"fi
```

✓ 1(ii) Factorial (Shell)

```
echo "Enter number:"read nfact=1i=1while [ $i -le $n ]do fact=$((fact * i)) i=$((i + 1))doneecho "Factorial = $fact"
```

✓ 2. Round Robin

```
#include<stdio.h>int main(){    int n, tq, time=0, bt[10], rem[10];    printf("Enter number of processes: ");    scanf("%d",&n);    printf("Enter burst times:\n");    for(int i=0;i<n;i++){        scanf("%d",&bt[i]);    }    printf("Enter time quantum: ");    scanf("%d",&tq);    while(1){        int done = 1;        for(int i=0;i<n;i++){            if(rem[i] > 0){                done = 0;                if(rem[i] > tq){                    time += tq;                    rem[i] -= tq;                } else{                    time += rem[i];                    rem[i] = 0;                }            }            if(done) break;        }        printf("Total Time = %d", time);    }
```

Output

```
Enter n: 3Enter burst times:5 3 1Enter time quantum: 2Total Time=9
```

✓ 4. FIFO Page Replacement

```
#include <stdio.h>

int main() {
    int frames, pages, i, j, k, flag, page_faults = 0;

    printf("Enter number of frames: ");
    scanf("%d", &frames);

    printf("Enter number of pages: ");
    scanf("%d", &pages);
```

```

int ref[pages], frame[frames];

printf("Enter reference string: ");
for(i = 0; i < pages; i++) {
    scanf("%d", &ref[i]);
}

// Initialize frames to -1
for(i = 0; i < frames; i++) {
    frame[i] = -1;
}

int index = 0;

printf("\nPage\tFrames\n");

for(i = 0; i < pages; i++) {
    flag = 0;

    // Check if page already in frame
    for(j = 0; j < frames; j++) {
        if(frame[j] == ref[i]) {
            flag = 1;
            break;
        }
    }

    // If page fault
    if(flag == 0) {
        frame[index] = ref[i];
        index = (index + 1) % frames;
        page_faults++;
    }

    // Print frames
    printf("%d\t", ref[i]);
    for(k = 0; k < frames; k++) {
        if(frame[k] != -1)
            printf("%d ", frame[k]);
        else
            printf("- ");
    }
    printf("\n");
}

printf("\nTotal Page Faults = %d\n", page_faults);

```

```
    return 0;
}
```

✓ Sample Input

```
Enter number of frames: 3
Enter number of pages: 12
Enter reference string: 1 2 3 4 1 2 5 1 2 3 4 5
```

✓ Sample Output

Page	Frames
1	1 - -
2	1 2 -
3	1 2 3
4	4 2 3
1	4 1 3
2	4 1 2
5	5 1 2
1	5 1 2
2	5 1 2
3	5 3 2
4	5 3 4
5	5 3 4

Total Page Faults = 9

Here's a **simple C program to implement the Paging technique** for memory management (logical → physical address translation), along with a sample output.

✓ C Program (Paging Technique)

```
#include <stdio.h>

int main() {
    int p, f, i;

    printf("Enter number of pages: ");
```

```

scanf("%d", &p);

printf("Enter number of frames: ");
scanf("%d", &f);

int page_table[p];

printf("Enter page table (frame number for each page):\n");
for(i = 0; i < p; i++) {
    printf("Page %d -> Frame: ", i);
    scanf("%d", &page_table[i]);
}

int logical_address, page_size;

printf("Enter page size: ");
scanf("%d", &page_size);

printf("Enter logical address: ");
scanf("%d", &logical_address);

int page_no = logical_address / page_size;
int offset = logical_address % page_size;

if(page_no >= p) {
    printf("Invalid Page Number!\n");
} else {
    int frame_no = page_table[page_no];
    int physical_address = frame_no * page_size + offset;

    printf("\nPage Number = %d\n", page_no);
    printf("Offset = %d\n", offset);
    printf("Frame Number = %d\n", frame_no);
    printf("Physical Address = %d\n", physical_address);
}

return 0;
}

```

Sample Input

```

Enter number of pages: 4
Enter number of frames: 4
Enter page table (frame number for each page):
Page 0 -> Frame: 2
Page 1 -> Frame: 1

```

```
Page 2 -> Frame: 3
Page 3 -> Frame: 0
Enter page size: 1024
Enter logical address: 2050
```

✓ Sample Output

```
Page Number = 2
Offset = 2
Frame Number = 3
Physical Address = 3074
```

💡 Quick Explanation (for viva)

- Logical address = **Page number + Offset**
 - Page number = Logical Address / Page Size
 - Offset = Logical Address % Page Size
 - Physical Address = (Frame Number × Page Size) + Offset
 - Page table maps **pages** → **frames**
-

If you want, I can also give:

- ♦ Segmentation program
- ♦ Combined paging + segmentation
- ♦ Viva questions & answers for this topic
-

For OS practicals, “Threading and Synchronization” is usually demonstrated using the **Producer–Consumer problem** with threads and semaphores.

Here’s a **simple, standard C program** using **POSIX threads** (`pthread`) and **semaphores**.

✓ C Program (Threading + Synchronization using Producer–Consumer)

```
#include <stdio.h>
#include <pthread.h>
```

```

#include <semaphore.h>
#include <unistd.h>

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

// Producer function
void* producer(void* arg) {
    int item;
    for(int i = 0; i < 10; i++) {
        item = i + 1;

        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        sleep(1);
    }
    return NULL;
}

// Consumer function
void* consumer(void* arg) {
    int item;
    for(int i = 0; i < 10; i++) {

        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        sleep(1);
    }
    return NULL;
}

```

```

}

int main() {
    pthread_t p, c;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}

```

✓ Sample Output

```

Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
...

```

(Order may vary slightly due to thread scheduling)

💡 Key Concepts (for viva)

- **Threading:** Multiple threads run concurrently using `pthread_create()`
- **Synchronization** ensures correct access to shared data
- **Mutex** (`pthread_mutex_t`) → prevents race conditions

- **Semaphores:**
 - `empty` → counts empty slots
 - `full` → counts filled slots
 - `sem_wait()` → decrease (wait)
 - `sem_post()` → increase (signal)
-

Compile & Run (important)

```
gcc program.c -o program -lpthread
./program
```

If you want, I can also give:

- ♦ Reader–Writer problem program
- ♦ Dining Philosophers
- ♦ Viva questions & answers for this experiment

Here's a **simple, exam-ready C program** to implement **Fixed Partition Memory Allocation using First Fit**, along with sample output.

C Program (First Fit – Fixed Partition)

```
#include <stdio.h>

int main() {
    int blockSize[20], processSize[20];
    int m, n, i, j;
    int allocation[20];

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    printf("Enter size of each block:\n");
    for(i = 0; i < m; i++) {
        printf("Block %d: ", i + 1);
        scanf("%d", &blockSize[i]);
    }

    printf("Enter number of processes: ");
    scanf("%d", &n);
```

```

printf("Enter size of each process:\n");
for(i = 0; i < n; i++) {
    printf("Process %d: ", i + 1);
    scanf("%d", &processSize[i]);
    allocation[i] = -1; // Not allocated initially
}

// First Fit Allocation
for(i = 0; i < n; i++) {
    for(j = 0; j < m; j++) {
        if(blockSize[j] >= processSize[i]) {
            allocation[i] = j;
            blockSize[j] = 0; // Block is now occupied (fixed partition)
            break;
        }
    }
}

// Display result
printf("\nProcess No.\tProcess Size\tBlock No.\n");
for(i = 0; i < n; i++) {
    printf("%d\t\t%d\t\t", i + 1, processSize[i]);
    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}

return 0;
}

```

Sample Input

```

Enter number of memory blocks: 5
Enter size of each block:
Block 1: 100
Block 2: 500
Block 3: 200
Block 4: 300
Block 5: 600

```

```

Enter number of processes: 4
Enter size of each process:
Process 1: 212
Process 2: 417

```

Process 3: 112
Process 4: 426

✓ Sample Output

Process No.	Process Size	Block No.
1	212	2
2	417	5
3	112	3
4	426	Not Allocated

💡 Quick Explanation (for viva)

- **First Fit:** Allocate the **first block that is large enough**
- Fixed partition → each block is used **only once**
- If no block fits → process is **not allocated**
- Fast but may cause **internal fragmentation**

Here's a **clean, exam-ready C program** to implement **Banker's Algorithm (Deadlock Avoidance)**, along with a sample output.

✓ C Program (Banker's Algorithm)

```
#include <stdio.h>

int main() {
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m], finish[n], safeSeq[n];

    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
```

```

        scanf("%d", &alloc[i][j]);
    }
}

printf("\nEnter Maximum Matrix:\n");
for(i = 0; i < n; i++) {
    for(j = 0; j < m; j++) {
        scanf("%d", &max[i][j]);
    }
}

printf("\nEnter Available Resources:\n");
for(i = 0; i < m; i++) {
    scanf("%d", &avail[i]);
}

// Calculate Need matrix
for(i = 0; i < n; i++) {
    for(j = 0; j < m; j++) {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

// Initialize finish array
for(i = 0; i < n; i++) {
    finish[i] = 0;
}

int count = 0;

while(count < n) {
    int found = 0;

    for(i = 0; i < n; i++) {
        if(finish[i] == 0) {
            for(j = 0; j < m; j++) {
                if(need[i][j] > avail[j])
                    break;
            }

            if(j == m) {
                for(k = 0; k < m; k++) {
                    avail[k] += alloc[i][k];
                }

                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
}

```

```

    }

    if(found == 0) {
        printf("\nSystem is NOT in safe state (Deadlock possible)\n");
        return 0;
    }
}

printf("\nSystem is in SAFE state\nSafe Sequence: ");
for(i = 0; i < n; i++) {
    printf("P%d ", safeSeq[i]);
}

printf("\n");
return 0;
}

```

✓ Sample Input

Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:

```

0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

```

Enter Maximum Matrix:

```

7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

```

Enter Available Resources:

```

3 3 2

```

✓ Sample Output

System is in SAFE state
Safe Sequence: P1 P3 P4 P0 P2

💡 Key Points (for viva)

- **Need = Max – Allocation**
 - A system is **safe** if a sequence exists where all processes can finish
 - Banker's Algorithm checks **safe state before allocation**
 - Prevents deadlock instead of detecting it later
-

✅ C Program (Producer–Consumer using Semaphores)

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

// Producer function
void* producer(void* arg) {
    int item;
    for(int i = 0; i < 10; i++) {
        item = i + 1;

        sem_wait(&empty);          // wait if buffer is full
        pthread_mutex_lock(&mutex); // enter critical section

        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&mutex); // exit critical section
        sem_post(&full);              // signal item added

        sleep(1);
    }
    return NULL;
}

// Consumer function
```

```

void* consumer(void* arg) {
    int item;
    for(int i = 0; i < 10; i++) {

        sem_wait(&full);           // wait if buffer is empty
        pthread_mutex_lock(&mutex); // enter critical section

        item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % SIZE;

        pthread_mutex_unlock(&mutex); // exit critical section
        sem_post(&empty);           // signal space available

        sleep(1);
    }
    return NULL;
}

int main() {
    pthread_t p, c;

    sem_init(&empty, 0, SIZE); // initially all slots empty
    sem_init(&full, 0, 0);     // initially no items
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}

```

Sample Output

```

Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3

```

```
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
...
```

(Output order may vary due to thread scheduling)

💡 Key Points (for viva)

- `empty` semaphore → counts empty buffer slots
- `full` semaphore → counts filled slots
- `mutex` → ensures mutual exclusion
- `sem_wait()` → decrement (wait)
- `sem_post()` → increment (signal)
- Avoids **race condition** and **buffer overflow/underflow**

⚙️ Compile & Run

```
gcc program.c -o program -lpthread
./program
```

Here's a **simple, exam-ready C program** to implement the **SJF (Shortest Job First) Scheduling Algorithm (Non-Preemptive)**, along with sample output.

✅ C Program (SJF – Non-Preemptive)

```
#include <stdio.h>

int main() {
    int n, i, j;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], wt[n], tat[n], p[n];

    // Input burst times
    printf("Enter Burst Time:\n");
```

```

for(i = 0; i < n; i++) {
    printf("P%d: ", i + 1);
    scanf("%d", &bt[i]);
    p[i] = i + 1;
}

// Sort processes by burst time (SJF)
for(i = 0; i < n - 1; i++) {
    for(j = i + 1; j < n; j++) {
        if(bt[i] > bt[j]) {
            int temp;

            temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
            temp = p[i]; p[i] = p[j]; p[j] = temp;
        }
    }
}

// Calculate waiting time
wt[0] = 0;
for(i = 1; i < n; i++) {
    wt[i] = wt[i - 1] + bt[i - 1];
}

// Calculate turnaround time
for(i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

// Display results
printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
for(i = 0; i < n; i++) {
    printf("P%d\t%d\t\t%d\t\t%d\n", p[i], bt[i], wt[i], tat[i]);
}

// Calculate averages
float avg_wt = 0, avg_tat = 0;
for(i = 0; i < n; i++) {
    avg_wt += wt[i];
    avg_tat += tat[i];
}

printf("\nAverage Waiting Time = %.2f", avg_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);

return 0;
}

```

✓ Sample Input

```
Enter number of processes: 4
Enter Burst Time:
P1: 6
P2: 8
P3: 7
P4: 3
```

✓ Sample Output

Process	Burst Time	Waiting Time	Turnaround Time
P4	3	0	3
P1	6	3	9
P3	7	9	16
P2	8	16	24

Average Waiting Time = 7.00
Average Turnaround Time = 13.00

💡 Key Points (for viva)

- SJF selects the process with **smallest burst time**
- This is **non-preemptive** (once started, it runs fully)
- Minimizes **average waiting time**
- Requires knowing burst time in advance

Here's a **simple, exam-ready C program** to implement **Inter Process Communication (IPC)** using pipes.

✓ C Program (IPC using Pipe)

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    char write_msg[] = "Hello from Parent using Pipe";
```

```
char read_msg[100];

// Create pipe
if (pipe(fd) == -1) {
    printf("Pipe failed\n");
    return 1;
}

if (fork() == 0) {
    // Child process (Reader)
    close(fd[1]); // close write end

    read(fd[0], read_msg, sizeof(read_msg));
    printf("Child received: %s\n", read_msg);

    close(fd[0]);
} else {
    // Parent process (Writer)
    close(fd[0]); // close read end

    write(fd[1], write_msg, strlen(write_msg) + 1);
    printf("Parent sent: %s\n", write_msg);

    close(fd[1]);
}

return 0;
}
```

✓ Sample Output

```
Parent sent: Hello from Parent using Pipe
Child received: Hello from Parent using Pipe
```

💡 Key Points (for viva)

- **Pipe** is a unidirectional IPC mechanism
- `pipe(fd)` creates two ends:
 - `fd[0]` → read end
 - `fd[1]` → write end
- `fork()` creates parent and child processes
- Parent writes → Child reads

- Must close unused ends to avoid errors

Compile & Run

```
gcc program.c -o program
./program
```

Here's a simple C program to implement the **LRU (Least Recently Used) Page Replacement Algorithm**:

```
#include <stdio.h>

int main() {
    int frames, pages, i, j, k, pos, faults = 0;

    printf("Enter number of frames: ");
    scanf("%d", &frames);

    printf("Enter number of pages: ");
    scanf("%d", &pages);

    int ref[pages], frame[frames], time[frames];

    printf("Enter page reference string:\n");
    for(i = 0; i < pages; i++) {
        scanf("%d", &ref[i]);
    }

    // Initialize frames and time
    for(i = 0; i < frames; i++) {
        frame[i] = -1;
        time[i] = 0;
    }

    int counter = 0;

    for(i = 0; i < pages; i++) {
        int found = 0;

        // Check if page already in frame (HIT)
        for(j = 0; j < frames; j++) {
            if(frame[j] == ref[i]) {
                counter++;
                time[j] = counter;
                found = 1;
                break;
            }
        }
    }
}
```

```

    }
}

// If not found (MISS)
if(!found) {
    faults++;

    // Find least recently used page
    int min = time[0];
    pos = 0;

    for(k = 1; k < frames; k++) {
        if(time[k] < min) {
            min = time[k];
            pos = k;
        }
    }

    frame[pos] = ref[i];
    counter++;
    time[pos] = counter;
}

// Display current frames
printf("Frame status: ");
for(j = 0; j < frames; j++) {
    if(frame[j] != -1)
        printf("%d ", frame[j]);
    else
        printf("- ");
}
printf("\n");
}

printf("\nTotal Page Faults = %d\n", faults);

return 0;
}

```

How it works:

- Uses an array `frame[]` to store current pages in memory.
- Uses `time[]` to track when each page was last used.
- On a **page hit**, updates its usage time.
- On a **page fault**, replaces the page with the **least recently used** one.

If you want, I can also provide:

- Step-by-step dry run
- Output example
- Optimized version using stack or linked list

Here's a **simple, exam-ready C program** to implement the **SSTF (Shortest Seek Time First) Disk Scheduling Algorithm**, along with sample output.

✓ C Program (SSTF Disk Scheduling)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, i, j, head, total_seek = 0;

    printf("Enter number of disk requests: ");
    scanf("%d", &n);

    int req[n], visited[n];

    printf("Enter disk request sequence:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &req[i]);
        visited[i] = 0;
    }

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("\nSeek Sequence: %d", head);

    for(i = 0; i < n; i++) {
        int min = 9999, index = -1;

        for(j = 0; j < n; j++) {
            if(!visited[j]) {
                int dist = abs(head - req[j]);
                if(dist < min) {
                    min = dist;
                    index = j;
                }
            }
        }

        visited[index] = 1;
        total_seek += min;
    }
}
```

```
        head = req[index];

        printf(" -> %d", head);
    }

    printf("\nTotal Seek Time = %d\n", total_seek);

    return 0;
}
```

✓ Sample Input

```
Enter number of disk requests: 8
Enter disk request sequence:
98 183 37 122 14 124 65 67
Enter initial head position: 53
```

✓ Sample Output

```
Seek Sequence: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Total Seek Time = 236
```

💡 Key Points (for viva)

- SSTF selects the request **closest to current head position**
- Reduces total seek time compared to FCFS
- May cause **starvation** (far requests may wait long)
- Uses **greedy approach** (minimum distance first)

⚙️ Compile & Run

```
gcc program.c -o program
./program
```

Here's a **simple and exam-ready C program** to implement the **LFU (Least Frequently Used) Page Replacement Algorithm**:

```

#include <stdio.h>

int main() {
    int frames, pages, i, j, pos, faults = 0;

    printf("Enter number of frames: ");
    scanf("%d", &frames);

    printf("Enter number of pages: ");
    scanf("%d", &pages);

    int ref[pages], frame[frames], freq[frames];

    printf("Enter page reference string:\n");
    for(i = 0; i < pages; i++) {
        scanf("%d", &ref[i]);
    }

    // Initialize frames and frequency
    for(i = 0; i < frames; i++) {
        frame[i] = -1;
        freq[i] = 0;
    }

    for(i = 0; i < pages; i++) {
        int found = 0;

        // Check if page is already present (HIT)
        for(j = 0; j < frames; j++) {
            if(frame[j] == ref[i]) {
                freq[j]++; // Increase frequency
                found = 1;
                break;
            }
        }

        // If page not found (MISS)
        if(!found) {
            faults++;

            // Find least frequently used page
            int min = freq[0];
            pos = 0;

            for(j = 1; j < frames; j++) {
                if(freq[j] < min) {
                    min = freq[j];
                    pos = j;
                }
            }
        }
    }
}

```

```

    }

    frame[pos] = ref[i];
    freq[pos] = 1; // Reset frequency for new page
}

// Display frame status
printf("Frame status: ");
for(j = 0; j < frames; j++) {
    if(frame[j] != -1)
        printf("%d ", frame[j]);
    else
        printf("- ");
}
printf("\n");
}

printf("\nTotal Page Faults = %d\n", faults);

return 0;
}

```

Key Points:

- `frame[]` → stores pages in memory
 - `freq[]` → stores how many times each page is used
 - On **hit** → increase frequency
 - On **fault** → replace page with **lowest frequency**
-

Sample Input:

```

Frames = 3
Pages = 7
Reference String = 1 2 3 1 2 4 5

```

Sample Output (shortened):

```

Frame status: 1 - -
Frame status: 1 2 -
Frame status: 1 2 3
Frame status: 1 2 3
Frame status: 1 2 3

```

```
Frame status: 4 2 3
Frame status: 5 2 3

Total Page Faults = 5
```

If you want, I can also give:

- LFU with **tie-breaking using LRU** (important for viva)
- Step-by-step table (like exam presentation)
- Comparison with FIFO & LRU

Here's a **simple, exam-ready C program** to implement the **SCAN (Elevator) Disk Scheduling Algorithm**, along with sample output.

✅ C Program (SCAN Disk Scheduling)

```
#include <stdio.h>

int main() {
    int n, i, j, head, disk_size, seek = 0;

    printf("Enter number of requests: ");
    scanf("%d", &n);

    int req[n];

    printf("Enter request sequence:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &req[i]);
    }

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("Enter disk size (max track number): ");
    scanf("%d", &disk_size);

    // Sort requests
    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(req[i] > req[j]) {
                int temp = req[i];
                req[i] = req[j];
                req[j] = temp;
            }
        }
    }

    // SCAN algorithm implementation
    while(seek < disk_size) {
        printf("Head is at track %d, moving towards track %d\n", head, seek);
        // Process requests at current head position
        for(i = 0; i < n; i++) {
            if(req[i] == head) {
                printf("Request %d at track %d\n", req[i], head);
            }
        }
        // Move head to next request
        if(seek < disk_size) {
            seek = req[i];
        }
        head = seek;
    }
}
```

```

    }
}

int index;
for(i = 0; i < n; i++) {
    if(head < req[i]) {
        index = i;
        break;
    }
}

printf("\nSeek Sequence: %d", head);

// Move towards higher end first
for(i = index; i < n; i++) {
    seek += (req[i] - head);
    head = req[i];
    printf(" -> %d", head);
}

// Move to end of disk
seek += (disk_size - 1 - head);
head = disk_size - 1;
printf(" -> %d", head);

// Reverse direction
for(i = index - 1; i >= 0; i--) {
    seek += (head - req[i]);
    head = req[i];
    printf(" -> %d", head);
}

printf("\nTotal Seek Time = %d\n", seek);

return 0;
}

```

Sample Input

```

Enter number of requests: 8
Enter request sequence:
98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size (max track number): 200

```

Sample Output

```
Seek Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 37 -> 14  
Total Seek Time = 331
```

Key Points (for viva)

- SCAN moves like an **elevator (one direction, then reverse)**
- Services all requests in one direction before reversing
- Reduces starvation compared to SSTF
- Also called **Elevator Algorithm**

Compile & Run

```
gcc program.c -o program  
./program
```